

Catman-do — Game Design Document

A Data-Driven, Extensible Cozy City Builder Optimized for Vibe-Coding

Project Name	Catman-do	Target Engine	Unity / Godot (3D with Isometric 2.5D Fallback)
Genre	Casual Town Builder / Management	Monetization	Premium / Buy-Once (No Microtransactions)
Architecture	Data-Driven JSON & Event System	Target Workflow	LLM Vibe-Coding & Modular Expansions

1. Core Architecture & Vibe-Coding Framework

To ensure smooth generation by AI coding assistants (vibe-coding), all entities, interactions, and rules are completely decoupled into Data-Driven Schemas and Event-Driven State Machines. The core engine never hardcodes building stats, cat traits, or layout bonuses.

- **Data-Driven Schemas:** All cats, mice, buildings, items, and recipe patterns exist as flat JSON files or engine ScriptableObjects.
- **3D / 2.5D Visual Pivot:** Built on a standard 3D isometric grid. If 3D mesh assets are unavailable during development, 2D billboarded sprites seamlessly swap into the same transform space with zero code alterations.
- **Microtransaction-Free Achievement Engine:** Unlocking achievements emits events that register against an `Unlocked_Content.json` registry, dynamically enabling new building options without external server dependencies.

2. Founder Cats (Starting Selection)

At game start, the player selects one of three Founder Cats. This choice establishes global multiplier flags and town personality for the run.

Founder Cat	Primary Pro	Primary Con	Aesthetic & Vibe
Barnaby The Mayor	+20% Mouse Recruitment Rate. Mice arrive with higher base morale.	-15% Construction Speed. Demands blueprint reviews before build phases.	Plump British Shorthair wearing a tiny hi-vis construction vest.
Whisper The Mystic	Night Phase Buffs. Catnip and crop yields double during night ticks.	+10% Mouse Pay Requirement. Mystical aura slightly spooks workers.	Sleek black cat with glowing eyes sitting in a meditative pose.
Turbo The Scrapper	+30% Resource Harvest. +25% player movement speed across grid.	5% Blueprint Failure Rate. Wastes 10% materials on construction failure.	Ginger tabby with a notched ear, twitching tail, and restless pacing.

3. Core Gameplay Loop

1. **Explore & Blueprint:** The Cat explores the fog of war, uncovers resource nodes (Wood, Twigs, Yarn), and places building blueprints on the grid.
2. **Recruit & Delegate:** The Cat recruits Mice — and, as of this revision, other animal species such as Cows — using species-appropriate recruitment costs and manages workforce assignments.
3. **Automate & Construct:** Recruited animals gather resources, construct buildings, and process products autonomously.
4. **Optimize & Discover:** The Cat arranges buildings into geometric layouts to trigger one-time Resonance Patterns and weather/wind synergies.

4. Mechanics Manual (12 Novel Casual Mechanics)

Mechanic	Gameplay Function	Vibe-Coding Logic
Architectural Resonance	Constructing buildings in specific geometric configurations yields unique permanent town buffs. Each pattern bonus can only be unlocked ONCE.	JSON spatial offset validator. Uses a runtime discovered_patterns hash set to enforce single-time activation.
Laser Pointer Prioritization	Clicking projects a red dot on the grid. Idle animals instantly pathfind toward the dot to prioritize work at that location.	AoE raycast signal that overrides default behavior trees with target priority.
Cardboard Tier & Rain Events	Cheap early buildings (shoe boxes, cartons) are fast to build but damaged by rain; must be protected by leaf canopies or upgraded to tin cans.	Weather event tick checking a building's waterproof boolean attribute.
Thermal Warmth Grid	Moving sunbeams and warm appliances emit heat tiles. Napping on warm tiles grants stamina and speed buffs.	Heat-map overlay system emitting localized thermal values to contiguous grid cells.
Cat-Nap Fast-Forward	Napping accelerates time by 5x into "Dream Mode," displaying ghost overlays of future pathfinding and supply bottlenecks.	Global time-scale multiplier (5.0x) paired with path-prediction gizmos and dreamy pastel screen shader.
Cheese-Standard Salary	Mice require specific cheese varieties. Over-aged cheese attracts pests; underpaying causes strikes. Other species use their own upkeep goods (e.g. Cows require feed).	Timer-based item degradation queue linked to happiness thresholds, generalized across animal species.
Whisker-Radar Hot/Cold	On-screen whiskers vibrate faster as the player cat approaches hidden rare resources in the fog of war.	Proximity-based distance calculation driving UI vibration oscillations.
Gravity "Cat-Stack" Scaffolds	Animals stack vertically to construct tall towers. High winds trigger a casual balancing slider mini-game.	Physics balance variable influenced by wind vectors during vertical construction state.
Mouse Guilds & Tiny Tools	Animals earn experience in job roles and equip craftable tools (Thimble Helmets, Button Shields).	Generic inventory array attached to individual animal data structures, shared across species.
Catnip Drift Dynamics	Prevailing winds carry catnip aroma, speeding up work downwind but lulling guard mice to sleep.	Grid-based vector wind calculator propagating scent intensity values.
Grooming & Dust Bunnies	Dust bunnies spawn at build sites. Swatting them grants Lint crafting material and boosts town morale.	Random entity spawner on active construction tiles with interactable loot drops.
Stray Cat Visitors	Wandering stray cats appear at map edges offering rare trades, challenges, or town membership choices. Attraction odds can be raised by keeping animals such as Cows that emit an attracts signal.	Random event controller initiating branching dialogue trees and trade tables, weighted by data-driven attracts fields.

5. JSON Definitions & Detection Logic

Pattern Configuration Schema (resonance_patterns.json)

The schema below demonstrates how geometric layouts are defined using localized coordinate offsets relative to an anchor building.

```
{
  "resonance_patterns": [
    {
      "id": "pattern_catnip_triangle",
      "display_name": "The Herbal Triad",
      "description": "A triangle of Mouse Huts focusing energy onto a Catnip Garden. (One-time bonus)",
      "max_stacks": 1,
      "anchor_building_type": "building_catnip_garden",
      "required_offsets": [
        { "building_type": "building_mouse_hut", "x_offset": 0, "z_offset": 2 },
        { "building_type": "building_mouse_hut", "x_offset": -2, "z_offset": -1 },
        { "building_type": "building_mouse_hut", "x_offset": 2, "z_offset": -1 }
      ],
      "bonuses": [
        {
          "target": "building_catnip_garden",
          "stat": "production_rate",
          "modifier_type": "multiplier",
          "value": 1.50
        },
        {
          "target": "global_mice",
          "stat": "movement_speed",
          "modifier_type": "additive",
          "value": 0.15
        }
      ]
    }
  ]
}
```

One-Time Bonus Enforcement Logic (Python / Pseudo-code)

This function strictly enforces that each pattern bonus can only be unlocked once per playthrough.

```
def on_building_constructed(new_building, save_data):
    potential_patterns = lookup_patterns_by_anchor(new_building.type)

    for pattern in potential_patterns:
        # RULE ENFORCEMENT: Check if this pattern was already discovered
        if pattern.id in save_data.discovered_patterns:
            continue # Skip; bonus already granted

        # Check grid offset matching across cardinal rotations
        if check_grid_offsets_match(new_building.position, pattern.required_offsets):
            save_data.discovered_patterns.append(pattern.id)
            apply_global_bonuses(pattern.bonuses)
```

```
trigger_discovery_ui(pattern.display_name)
break
```

6. Modular Expansion Architecture

To support ongoing updates and community mods without breaking base game logic, the engine uses a directory scanning architecture. Loading is two-pass: every expansion fragment is gathered and merged into the core registries first, and only then are cross-references (an animal's job output, a building's recipe inputs, etc.) validated — so a single expansion file can safely introduce a new animal and the new resource it produces together, in any order. Item, animal, and building ids are namespaced per mod (`core:milk`, `expansion_undersea:kelp`) to prevent collisions between expansions.

```
[Base Game Engine Core]
|
|→ [Loads Core JSON]
|   /Data/Core_Buildings.json
|   /Data/Core_Patterns.json
|   /Data/Core_Animal_Types.json
|
|→ [Scans Modular Directory] → /Expansions/
|   |→ Expansion_Undersea.json (Sub-aquatic tiles)
|   |→ Expansion_SpaceMice.json (Moon Cheese & Low-G)
```

7. Animal Recruitment & Husbandry System NEW

Mice were the only recruitable workforce in the original design. To support future species — starting with Cows, which produce Milk for cat happiness and for making Cheese — recruitment is generalized into a single species-agnostic framework rather than a mice-only system. This keeps the "no hardcoded entity logic" principle intact: a new animal is a data addition, not a new subsystem.

7.1 animal_types.json Schema

Each recruitable species is defined as data, not code:

- **id, display_name** — e.g. mouse, cow.
- **housing_building_type** — the building that must exist for this species to be recruitable (Mouse Hut, Barn).
- **movement_speed** — used by the shared pathfinding/state-machine code.
- **job_roles** — a list of jobs the species can perform, each with a duration and an output item id (e.g. Cow → produce_milk → core:milk).
- **upkeep** — what the species costs to keep happy (Mice: a cheese variety; Cows: feed and water).
- **attracts** — optional event-weight hooks other systems can read generically (see 7.3).

7.2 Unified Recruitment & Behavior

A single **Recruitment Board** replaces the mice-only Cheese Board: it lists whichever species currently have an available housing building on the map, and spends whatever upkeep cost that species' `animal_types.json` entry specifies. A single behavior state machine (*Idle* → *Moving to Job* → *Working* → *Idle*) drives every species; only the duration/output numbers it reads differ per species and job role. Laser Pointer Prioritization, morale, and strike mechanics all operate on this same shared pool regardless of species.

7.3 Worked Example: Cows

Adding Cows to an existing build requires no new manager, state machine, or UI screen — only:

1. A cow entry in `animal_types.json`: housing = Barn, a produce_milk job role, and feed/water upkeep.
2. A Milk entry in `items.json`, flowing through town storage exactly like any other good.
3. A Barn building (houses Cows) and a Creamery building whose produces field consumes Milk and outputs Cheese — closing the loop back into the existing Cheese-Standard Salary mechanic for Mice.
4. A happiness bonus applied when Milk is consumed by the Founder Cat, using the same {target, stat, modifier_type, value} bonus shape already used for Founder Cat traits and Resonance Pattern bonuses.
5. An attracts weight on the Cow's entry that the Stray Cat Visitors event controller reads to raise the chance of a stray sighting near a Barn.

8. Resource & Item Extensibility NEW

Resources (Wood, Twigs, Yarn, Cheese, Milk, and anything a future mod adds) are already data-driven via `items.json`, but two refinements make them properly modder-safe:

- **Open-ended item properties.** Every item carries a generic `properties` dictionary alongside its core fields (id, display name, icon, max stack, description). Mechanic-specific data — Cheese's variety and age for spoilage, a

hypothetical Wool's link back to the animal it's sheared from — lives in `properties` instead of requiring a new field on the core item schema every time a mechanic needs one more attribute.

- **Namespaced ids and two-pass loading.** Item, animal, and building ids are namespaced by source (`core:milk` vs. `expansion_x:wool`) so two independently authored expansions can't collide. The engine gathers every expansion's data fragments first, merges them, and only then validates cross-references — so one expansion file can introduce a new animal and the new resource it produces together, regardless of file load order.

Worked Example: A Modder Adding Wool

Add `expansion_x:wool` to the expansion's own items fragment with whatever `properties` its mechanics need; reference it from a new animal's `job_roles.output` and/or a building's `produces` field; ship the icon inside the expansion's own folder. No core script changes — the entire addition is data.